

AUDITORIA DE APLICAÇÃO WEB

Relatório de Auditoria de Segurança — ForceQA

Cinco categorias mapeadas para a stack real: RLS/tenant, gates de papel, IDOR nas rotas, segredos no código e XSS.

Data	28 de agosto de 2026
Projeto	ForceQA (repositório force-qa)
Escopo	Código-fonte do repositório force-qa (frontend React, APIs em api-src/, server.ts, migrations SQL em supabase/, configs de deploy Vercel). Sem Docker/Helm/Terraform/CI no repositório. Histórico git varrido por padrões de segredo. Bundle dist/ ausente no workspace (não rasterizado).
Método	Somente achados verificados no código (arquivo:linha). Os 10 handlers de API foram lidos por completo. Controles corretos estão na seção Pontos fortes.

Nota metodológica — mapeamento das categorias

- 1. Banco sem tranca → RLS Postgres (mecanismo de isolamento do projeto: organization_id + is_room_member() / can_admin_org()); superadmin atravessa). Não há ORM; o client usa PostgREST.
- 2. Permissão no navegador → cruzar src/lib/permissions.ts e gates em App.tsx com requireAdmin / requireSuperadmin / RLS.
- 3. IDOR → todos os handlers em api-src/ e server.ts, mais policies de UPDATE por id.
- 4. Chaves expostas → código, .env.example, vercel.json, git log, ausência de Docker/Helm/CI. Anon key no Vite é esperada.
- 5. XSS → React 19; busca de dangerouslySetInnerHTML/eval; href/src de usuário; e-mails só via Supabase Auth (sem HTML próprio).

1. Resumo executivo

Foram confirmados 14 achados no código: 1 crítica, 5 alta, 4 média, 2 baixa e 2 informativa. O risco central é a policy `users_update` permitir auto-promoção a superadmin (F-01), o que zera o isolamento multi-tenant. Em seguida, guest sem autenticação na validação de sala e storage público de evidências.

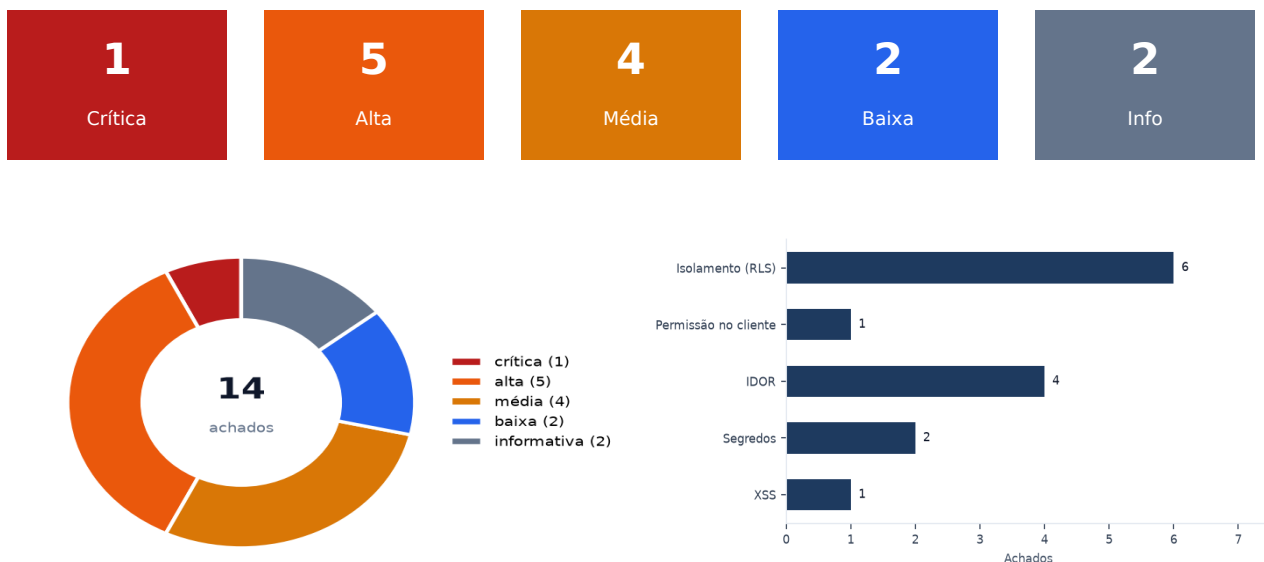


Figura 1 — Distribuição por severidade (rosca) e por categoria adaptada à stack (barras).

Stack detectada

- Frontend: React 19 + Vite 6 + TypeScript + Tailwind 4 (src/).
- Auth: Supabase Auth (JWT Bearer nas APIs). Anon key no browser.
- Banco / isolamento: Postgres + RLS. Tenant = `users.organization_id`. Superadmin = `users.is_superuser`.
- Acesso a dados: PostgreSQL via `@supabase/supabase-js`; `service_role` só no servidor (`getSupabaseAdmin()`). Sem ORM.
- Backend: 10 rotas POST em `api-src/` (Vercel) espelhadas em `server.ts` no dev.
- Deploy: Vercel (`vercel.json`). Sem Docker, Helm, Terraform ou GitHub Actions neste repositório.
- SQL: `supabase/00_run_order.sql` itens 1-13; isolamento efetivo na migration 11.

Não há Docker/Helm/Terraform/CI neste repo (categoria de defaults em charts não se aplica). E-mails de convite passam pelo Auth da Supabase, sem HTML próprio. Não há `dist/` no workspace; a análise das VITE_* foi na fonte. Revisão estática — sem teste dinâmico em produção.

2. Pontos fortes e pontos fracos

Pontos fortes (verificados)

Estes controles foram lidos no código e estão corretos no recorte indicado. Servem como prova de cobertura da auditoria, não como ausência de risco residual.

- ✓ APIs admin validam privilégio no servidor

`requireAdmin` em `create-user` e `delete-user`; `requireSuperadmin` em `create-organization` e `move-user` (`api-src/shared/auth.ts:55-68` e handlers em `api-src/admin/`). A UI (`App.tsx:391-395`) esconde as telas, e o backend repete o gate.

- ✓ `delete-user` recusa outra org e o próprio superadmin

`api-src/shared/adminUsers.ts:270-289` checa `ator` $\neq$ `alvo`, `is_superuser` do `alvo` e `organization_id`.

✓ Convite de sala checa papel, org e membership

inviteToRoom (rooms.ts:119–147): só admin/qa/scrum_master ou superadmin; recusa sala de outra org; QA/SM precisa ser membro.

✓ Join de usuário não-guest exige org + membership

rooms.ts:69–88 devolve 403 se a sala é de outra organização ou se não há linha em room_members. (O furo é só o ramo guest.)

✓ protect_user_role impede auto-promover role=admin

migration_create_organization_admin.sql:13–27. Não cobre is_superadmin (F-01), mas o papel admin via coluna role está travado.

✓ Signup não promove admin via user_metadata

handle_new_auth_user só aceita admin a partir de app_metadata (não gravável pelo cliente). user_metadata cai na lista non_admin_roles.

✓ SELECT de notificações preso ao dono

notifications_select USING (user_id = auth.uid()) em migration_access_and_security.sql:466–468.

✓ Bugs/salas/projetos SELECT via is_room_member (pós-migration 11)

A função is_room_member foi reescrita em organization_scope.sql:51–73 para exigir a mesma org (admin da org vê as salas; superadmin atravessa). war_rooms_select / bugs_select / projects_select continuam usando essa função — isolamento de leitura ok se a migration 11 rodou.

✓ Mutações de organizations só superadmin

organizations_insert/update/delete WITH CHECK is_superadmin (organization_scope.sql:146–157). renameOrganization no client depende disso.

✓ Sem segredos hardcoded nem defaults \${VAR:-secret}

Service role / Gemini / OpenRouter vêm de process.env; getSupabaseAdmin lança se a key falta (auth.ts:10–15). .gitignore ignora .env*. Git log não mostrou JWT de service_role real.

✓ XSS clássico de HTML evitado no React

Zero dangerouslySetInnerHTML / innerHTML / eval / new Function. Comentários e títulos interpolados como texto. Relatório IA (AIReportModal.tsx:182) renderiza markdown como texto (whitespace-pre-wrap), não como HTML. Sem lib de sanitização — e, nos pontos de HTML, não foi necessária. O gap é URL em href (F-10).

✓ Frontend congela campos privilegiados no updateProfile

AuthContext.tsx:386–396 força role, organizationId e isSuperadmin do perfil já carregado. Defesa na UI; insuficiente contra PostgREST direto (F-01).

Pontos fracos (riscos centrais)

- O isolamento multi-tenant inteiro depende de is_superadmin e organization_id na tabela users, mas o próprio usuário pode gravar os dois (F-01, F-02).
- Guest é um caminho intencional de produto que atravessa o tenant e ainda enumera salas sem login (F-04).
- Evidências de QA (screenshots) não são um segredo por sessão — o bucket é público (F-05).
- Gates de admin nas APIs próprias estão bons; o furo de privilégio está no PostgREST direto, que o frontend usa para quase tudo.
- XSS clássico está controlado; o gap é URL em href (F-10).

Cobertura IDOR — handlers de API

Todas as rotas em api-src/ (15 arquivos TypeScript, 10 handlers HTTP) foram lidas. Mutações de Kanban/bugs/salas no client passam só por RLS (não há handler próprio).

Rota	Auth / posse
POST /api/admin/create-user	requireAdmin + org do ator (superadmin pode escolher org)
POST /api/admin/create-organization	requireSuperadmin
POST /api/admin/delete-user	requireAdmin + posse de org + bloqueio de superadmin
POST /api/admin/move-user	requireSuperadmin
POST /api/rooms/invite	requireUser + papel + org da sala + membership se não admin
POST /api/rooms/join	requireUser; não-guest: org + membership; guest: F-04
POST /api/guest/validate-room	sem auth — F-04
POST /api/ai/suggest-bug-fields	requireUser apenas — F-09
POST /api/ai/detect-duplicate	requireUser apenas — F-09
POST /api/ai/generate-report	requireUser; metrics do body — F-09

3. Achados detalhados por categoria

Tabela-índice. O detalhe arquivo:linha, trecho e exploração segue na sequência, agrupado por categoria.

Sev.	ID	Arquivo:linha	Descrição
CRÍTICA	F-01	supabase/migration_organization_scope.sql:124-127	Qualquer autenticado pode promover a si mesmo a superadmin
ALTA	F-02	supabase/migration_organization_scope.sql:124-127	Troca de organization_id na própria linha (hop de tenant)
ALTA	F-03	supabase/migration_create_organization_admin.sql:45-74	Trigger de signup confia em user_metadata (org e papel)
ALTA	F-04	api-src/guest/validate-room.ts + api-src/shared/rooms.ts:validate-room.ts:12-14; rooms.ts:23-41, 69-93	Sala enumerável sem auth; guest entra e herda a org da sala
ALTA	F-05	supabase/migration_access_and_security.sql:491-511	Bucket evidence público e SELECT liberado a TO public
ALTA	F-06	supabase/migration_access_and_security.sql:413-415	comments_update não exige membership da sala destino
MÉDIA	F-07	api-src/rooms/invite.ts + server.ts:invite.ts:14-23; server.ts:115-123	redirectTo do convite copia o header Origin
MÉDIA	F-08	supabase/migration_access_and_security.sql:513-518	Upload no bucket evidence sem amarrar path ao tenant/sala
MÉDIA	F-09	api-src/ai/generate-report.ts + suggest-bug-fields.ts + detect-duplicate.ts:generate-report.ts:200-203; suggest:12; detect:12	Rotas de IA só exigem usuário autenticado
MÉDIA	F-10	src/components/BugDetailModal.tsx:390-392	href de evidência sem allowlist de esquema
BAIXA	F-11	supabase/migration_create_organization_admin.sql:4	GRANT amplo em organizations para o papel anon
BAIXA	F-12	src/lib/services.ts:247-290	warRoomId do activity log vem do cliente

Sev.	ID	Arquivo:linha	Descrição
INFORMATIVA	F-13	.env.example + supabase/schema.sql:.env.example:2; schema.sql (comentário de projeto)	URL real do projeto Supabase versionada
INFORMATIVA	F-14	src/lib/supabase.ts:17-40	Anon key embutida no bundle (modelo Supabase) e fallback placeholder

Categoria: Isolamento (RLS)

F-01 — Qualquer autenticado pode promover a si mesmo a superadmin

CRÍTICA

Onde: supabase/migration_organization_scope.sql:124-127

Por que é explorável: Sessão autenticada (viewer/guest inclusive) + chave anon (pública). Não depende de UI; o frontend até congela isSuperadmin em updateProfile, mas o cliente supabase-js no browser fala direto com PostgREST.

A policy users_update permite UPDATE da própria linha (id = auth.uid()) sem restrição de colunas. O trigger protect_user_role() só trava a coluna role — não is_superadmin. Um PATCH PostgREST SET is_superadmin=true escala o atacante a todas as organizações.

Trecho

```
CREATE POLICY "users_update" ON public.users
  FOR UPDATE TO authenticated
  USING (id = auth.uid() OR public.can_admin_org(organization_id))
  WITH CHECK (id = auth.uid() OR public.can_admin_org(organization_id));
```

F-02 — Troca de organization_id na própria linha (hop de tenant)

ALTA

Onde: supabase/migration_organization_scope.sql:124-127

Por que é explorável: Precisa de um UUID de org válido (o default 11111111-... está no código; outros vazam no perfil após um join de guest). Depois o atacante lê PII do tenant alvo via SELECT em users.

A mesma policy users_update autoriza o dono da linha a gravar qualquer organization_id. users_select devolve todos os perfis da org alvo (e-mail, nome, papel). O trigger não protege organization_id.

Trecho

```
USING (id = auth.uid() OR public.can_admin_org(organization_id))
WITH CHECK (id = auth.uid() OR public.can_admin_org(organization_id));
```

F-03 — Trigger de signup confia em user_metadata (org e papel)

ALTA

Onde: supabase/migration_create_organization_admin.sql:45-74

Por que é explorável: Explorável enquanto o Auth permitir signUp — o fluxo de convidado (AuthContext.tsx:268) prova que o cadastro público está habilitado. A UI de login não oferece signup, mas a anon key no browser basta.

handle_new_auth_user() lê role e organization_id de raw_user_meta_data, controlado pelo cliente no signUp. Papéis não-admin (qa, scrum_master, ...) são aceitos. Se o UUID da org existir, o usuário nasce dentro do tenant.

Trecho

```
user_role := NULLIF(TRIM(NEW.raw_user_meta_data->>'role'), '');
...
ELSIF user_role = ANY(non_admin_roles) THEN
  assigned_role := user_role;
org_id := NULLIF(TRIM(NEW.raw_user_meta_data->>'organization_id'), '')::uuid;
COALESCE(org_id, public.default_organization_id())
```

Categoria: IDOR

F-04 — Sala enumerável sem auth; guest entra e herda a org da sala

ALTA

Onde: api-src/guest/validate-room.ts + api-src/shared/rooms.ts:validate-room.ts:12-14; rooms.ts:23-41, 69-93

Por que é explorável: Produto exige link da sala, mas o nome (ILIKE) e a ausência de auth transformam isso em enumeração. Depois o guest autentica (signUp) e entra na org da vítima. can_guest_join_room() também não filtra tenant.

POST /api/guest/validate-room não chama requireUser. Com service_role, busca sala por id ou ILIKE no nome e devolve {id, name}. 404 vs 403 ainda confirma existência. joinRoom, para isGuest=true, pula o check de org e grava users.organization_id = roomOrgId via service_role.

Trecho

```
# validate-room.ts - sem requireUser
const result = await validateGuestRoom(...);
admin.from("war_rooms").select("id, name, ...").ilike("name", trimmed)
# joinRoom guest:
if (roomOrgId) {
  await admin.from("users").update({ organization_id: roomOrgId }).eq("id", userId);
}
```

F-05 — Bucket evidence público e SELECT liberado a TO public

ALTA

Onde: supabase/migration_access_and_security.sql:491-511

Por que é explorável: Sem autenticação. Condição: migration_access_and_security.sql aplicada (está no run order). O client usa getPublicUrl — o segredo é o path, não a sessão.

O bucket é public=true e a policy evidence_select permite SELECT a qualquer um (anon). Quem listar storage.objects lê todas as evidências de todos os tenants. URL pública também basta se o path vazar.

Trecho

```
INSERT INTO storage.buckets (... , public, ...)
VALUES ('evidence', 'evidence', true, ...)
CREATE POLICY "evidence_select" ON storage.objects
FOR SELECT TO public
USING (bucket_id = 'evidence');
```

F-06 — comments_update não exige membership da sala destino

ALTA

Onde: supabase/migration_access_and_security.sql:413-415

Por que é explorável: Autenticado que já comentou em alguma sala + UUID/id da sala alvo (vazável por F-04). Injeção de conteúdo entre tenants.

UPDATE em bug_comments só exige user_id = auth.uid(). Sem WITH CHECK de is_room_member, o USING vale para a linha nova: o dono do comentário pode mudar war_room_id (e bug_id) para qualquer sala. Membros da sala alvo passam a ver o texto (comments_select usa is_room_member).

Trecho

```
CREATE POLICY "comments_update" ON public.bug_comments
  FOR UPDATE TO authenticated
  USING (user_id = auth.uid());
```

F-07 — redirectTo do convite copia o header Origin

MÉDIA

Onde: api-src/rooms/invite.ts + server.ts:invite.ts:14-23; server.ts:115-123

Por que é explorável: Requer papel admin/qa/scrum_master (API valida isso). Não é IDOR de objeto; é open redirect no fluxo de convite.

O e-mail de convite (inviteUserByEmail) usa redirectTo montado com req.headers.origin. Origin é controlado pelo cliente. Um QA/admin autenticado pode apontar o magic link para um domínio de phishing.

Trecho

```
const origin = String(req.headers.origin || process.env.APP_URL || "https://force-qa.vercel.app");
redirectTo: `${origin.replace(/\$/ , "")}/` ,
```

F-08 — Upload no bucket evidence sem amarrar path ao tenant/sala

MÉDIA

Onde: supabase/migration_access_and_security.sql:513-518

Por que é explorável: Papel ≠ viewer. O client (evidence.ts:39) usa `\${roomId}/uuid.ext` mas o Storage não valida isso.

evidence_insert só checa can_write_bugs(). Qualquer não-viewer pode gravar em qualquer prefixo (ex.: pasta de outra sala). Combinado com F-05, o objeto fica publicamente legível.

Trecho

```
CREATE POLICY "evidence_insert" ON storage.objects
  FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'evidence' AND public.can_write_bugs());
```

Categoria: Permissão no cliente

F-09 — Rotas de IA só exigem usuário autenticado

MÉDIA

Onde: api-src/ai/generate-report.ts + suggest-bug-fields.ts + detect-duplicate.ts:generate-report.ts:200-203; suggest:12; detect:12

Por que é explorável: Abuso de cota (Gemini/OpenRouter). Não lê dados de outro tenant no banco — o JSON vem do cliente. Guest da F-04 também dispara IA.

requireUser sem papel, sem membership de sala e sem rate limit. Viewer/guest autenticado chama as três rotas. generate-report aceita metrics no body (não lê o board no servidor). A UI esconde criação de card para viewer, mas a aba Relatório IA fica visível para qualquer membro (WarRoomDetail.tsx:504-514).

Trecho

```
await requireUser(req.headers.authorization);
const result = await generateExecutiveReport(body.metrics);
```

Categoria: XSS

F-10 — href de evidência sem allowlist de esquema

MÉDIA

Onde: src/components/BugDetailModal.tsx:390-392

Por que é explorável: Quem pode escrever bugs (can_write_bugs) grava evidence_url=javascript:... Outro membro clica. XSS armazenado no mesmo tenant. updateBugField (services.ts:267) não valida URL.

evidenceUrl controlado pelo usuário vira href direto. isImageEvidence / isHttpEvidence não rejeitam javascript: — evidenceLabel cai em "link" e o âncora é renderizado. Sem DOMPurify no package.json. React escapa texto, mas não sanitiza URLs em href.

Trecho

```
<a href={activeBug.evidenceUrl} target="_blank" rel="noopener noreferrer" ...>
```

F-11 — GRANT amplo em organizations para o papel anon

BAIXA

Onde: supabase/migration_create_organization_admin.sql:4

Por que é explorável: Não explorável com as policies atuais. Endurecimento.

GRANT SELECT, INSERT, UPDATE, DELETE TO anon, authenticated, service_role. Policies de organizations são TO authenticated + is_superuser; anon sem policy RLS é deny. Superfície extra se uma policy futura for frouxa.

Trecho

```
GRANT SELECT, INSERT, UPDATE, DELETE ON public.organizations TO anon, authenticated, service_role;
```


F-12 — warRoomId do activity log vem do cliente

BAIXA

Onde: src/lib/services.ts:247-290

Por que é explorável: Integridade no mesmo tenant. RLS impede org alheia.

updateBugField atualiza o bug por id (RLS da sala do bug) e grava o log com o warRoomId enviado pelo browser. logs_insert exige is_room_member desse id — não cruza tenant, mas permite log enganoso em outra sala da mesma org da qual o usuário já é membro.

Trecho

```
await supabase.from("bugs").update(payload).eq("id", bugId);
await createActivityLog({ bugId, warRoomId, userId, ... });
```

Categoria: Segredos

F-13 — URL real do projeto Supabase versionada

INFORMATIVA

Onde: .env.example + supabase/schema.sql:env.example:2; schema.sql (comentário de projeto)

Por que é explorável: Reconhecimento; a anon key de produção está no bundle Vite (esperado).

VITE_SUPABASE_URL aponta para bdvpzgrgwgcvfgflelbn.supabase.co. Não é service_role. Anon/service_role no example são placeholders. Histórico git não contém eyJ de service_role nem Alza/sk_live reais.

Trecho

```
VITE_SUPABASE_URL="https://bdvpzgrgwgcvfgflelbn.supabase.co"
VITE_SUPABASE_ANON_KEY="SUA_ANON_KEY_AQUI"
SUPABASE_SERVICE_ROLE_KEY="SUA_SERVICE_ROLE_KEY_AQUI"
```

F-14 — Anon key embutida no bundle (modelo Supabase) e fallback placeholder

INFORMATIVA

Onde: src/lib/supabase.ts:17-40

Por que é explorável: N/A — comportamento esperado. Anon + F-01 é que vira arma.

Vite embute VITE_* no JS. A anon key é pública por desenho. Fallbacks https://placeholder.supabase.co e "placeholder" não são segredos. getSupabaseAdmin() exige SUPABASE_SERVICE_ROLE_KEY no servidor e falha sem default. Nenhum \${VAR:-segredo} no repo. Sem Docker/CI/Helm.

Trecho

```
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL as string;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY as string;
createClient(supabaseUrl || "https://placeholder.supabase.co", supabaseAnonKey || "placeholder")
```

4. Recomendações priorizadas

Pri.	Alvo	Ação
P1	F-01 / F-02	Travar is_superuser e organization_id no banco (trigger + revoke de UPDATE amplo). Isso sozinho impede a queda total do modelo de tenants.
P1	F-04	Autenticar ou tokenizar validate-room; guest não deve herdar organization_id. Remover busca por nome (ILIKE).
P1	F-05 / F-08	Bucket evidence privado, policies por prefixo de sala, URLs assinadas.
P1	F-03	Trigger de signup ignora user_metadata de org/role; guest só via API server-side.

Pri.	Alvo	Ação
P2	F-06	WITH CHECK em comments_update com is_room_member e war_room_id imutável.
P2	F-07	redirectTo allowlist (APP_URL); nunca Origin.
P2	F-09	requireAdmin/can_write_bugs + rate limit; relatório gera métricas no servidor.
P2	F-10	Allowlist https nas URLs de evidência + CHECK no Postgres.
P3	F-11 / schema.sql	REVOKE anon; não deixar policies USING(true) no bootstrap.
P3	F-12	Log de atividade deriva war_room_id do bug, não do client.
P3	Operacional	Confirmar no dashboard Supabase que as migrations 8-13 rodaram; schema.sql sozinho é inseguro.

5. Issues para o GitHub

Cada bloco abaixo está pronto para colar numa issue. Achados triviais do mesmo tema foram agrupados para não gerar spam. Copie do delimitador --- ISSUE n --- até --- FIM ISSUE n ---.

Issue 1 — [Segurança] Escalação a superadmin e troca de organização via UPDATE em users · labels: security, severity:critical

```
--- ISSUE 1 ---

# [Segurança] Escalação a superadmin e troca de organização via UPDATE em users

**Labels sugeridas:** `security, severity:critical`

**Achados:** F-01, F-02

## Problema
A policy RLS `users_update` autoriza qualquer autenticado a atualizar a própria linha sem restrição de colunas. O trigger `protect_user_role()` só protege a coluna `role`. Com a anon key (pública no frontend) o atacante chama PostgREST e grava `is_superadmin = true` e/ou `organization_id` de outro tenant.

O frontend (`AuthContext.updateProfile`) congela esses campos na UI — isso não é controle de acesso.

## Por que é explorável
- Policy USING/WITH CHECK: `id = auth.uid() OR can_admin_org(...)`
- Nenhuma column policy / trigger em `is_superadmin` ou `organization_id`.
- Cliente supabase-js no browser fala com `/rest/v1/users`.

## Evidência
`supabase/migration_organization_scope.sql:124-127`
```sql
CREATE POLICY "users_update" ON public.users
FOR UPDATE TO authenticated
USING (id = auth.uid() OR public.can_admin_org(organization_id))
WITH CHECK (id = auth.uid() OR public.can_admin_org(organization_id));
```
`supabase/migration_create_organization_admin.sql:6-29` — trigger só compara `NEW.role`.

## Impacto
- Crítico: qualquer login vira superadmin (lê/escreve todas as orgs, cria orgs, move usuários).
- Alto: hop de tenant e leitura de e-mails/nomes da org alvo (`users_select`).

## Sugestão de correção
1. Trigger `protect_user_privileges`: se `is_superadmin` ou `organization_id` mudarem e o caller não for superadmin (via `auth.uid()` + flag antiga da linha, ou service_role), `RAISE EXCEPTION`.
2. Alternativa: policy de UPDATE só para `name`, `avatar_url`, `squad` via view atualizável / `GRANT UPDATE (colunas)` e revogar UPDATE amplo.
3. `is_superadmin` só mutável por service_role (APIs admin).

## Critérios de aceite
- [ ] `UPDATE users SET is_superadmin = true WHERE id = auth.uid()` com JWT de viewer falha.
- [ ] `UPDATE users SET organization_id = '<outra>' WHERE id = auth.uid()` com JWT de viewer falha.
- [ ] Superadmin / API service_role ainda consegue promover e mover usuários.
- [ ] `UPDATE` do próprio `name`/`avatar_url` continua permitido.

--- FIM ISSUE 1 ---
```

Issue 2 — [Segurança] Trigger de signup aceita organization_id e role do cliente · labels: security, severity:high

--- ISSUE 2 ---

[Segurança] Trigger de signup aceita organization_id e role do cliente

****Labels sugeridas:**** `security, severity:high`

****Achados:**** F-03

Problema

`handle\_new\_auth\_user` lê `raw\_user\_meta\_data.role` e `organization\_id`. O cliente controla esses campos no `signup`. Papéis qa/scrum_master/etc. são aceitos. Se o UUID da org existir, o usuário entra no tenant sem convite.

Por que é explorável

O fluxo de convidado chama `supabase.auth.signup` (`src/context/AuthContext.tsx:268`), então o cadastro Auth está habilitado. A tela de login não precisa oferecer signup: a anon key basta no DevTools.

Evidência

```
`supabase/migration_create_organization_admin.sql:45-74`
```sql
user_role := NULLIF(TRIM(NEW.raw_user_meta_data->>'role'), '');
ELSIF user_role = ANY(non_admin_roles) THEN
assigned_role := user_role;
org_id := NULLIF(TRIM(NEW.raw_user_meta_data->>'organization_id'), '')::uuid;
-- INSERT ... COALESCE(org_id, public.default_organization_id())
```
```

Impacto

Conta QA/SM na org da vítima (UUID default é público; outros vazam após guest join). Bypass do modelo convite-only.

Sugestão de correção

- Ignorar `organization\_id` e `role` de `user\_metadata` no trigger. Sempre `viewer` + org default (ou null até o convite).
- Só gravar org/role via `app\_metadata` setada por service_role (já usado em `inviteUserByEmail` / admin create-user).
- Desabilitar signup público no dashboard se o produto for convite-only; guest deve usar um endpoint server-side.

Critérios de aceite

- [] `signup({ data: { organization\_id: '<org vítima>', role: 'qa' } })` resulta em viewer na org default.
- [] Convite admin e create-user continuam aplicando org/role corretos.
- [] Guest não escolhe org no metadata (só o servidor).

--- FIM ISSUE 2 ---

Issue 3 — [Segurança] Validação de sala sem autenticação e guest herda a organização · labels: security, severity:high

```
--- ISSUE 3 ---

# [Segurança] Validação de sala sem autenticação e guest herda a organização

**Labels sugeridas:** `security, severity:high`

**Achados:** F-04

## Problema
`POST /api/guest/validate-room` não autentica e, com service_role, resolve sala por **id ou nome (ILIKE)** devolvendo `{id, name}`. 403 (guest desligado) vs 404 confirma existência. `joinRoom` no ramo guest **não** checa organização e faz `users.organization_id = room.organization_id`.

## Por que é explorável
1. Enumerar nomes de salas sem login.
2. Criar guest (`signUp`) e `POST /api/rooms/join` – entra na org da sala.

## Evidência
`api-src/guest/validate-room.ts:12-14` – sem `requireUser`.
`api-src/shared/rooms.ts:23-41` (lookup ilike + 403/404).
`api-src/shared/rooms.ts:69-93` (guest pula org check e atualiza `organization_id`).
`server.ts:88-94` espelha a rota.

`can_guest_join_room` (`migration_access_and_security.sql:132-145`) também não filtra tenant.

## Impacto
Acesso de leitura ao Kanban/bugs/comentários da org alvo (admin da org vê o guest em `users`).
PII de colegas via `users_select`.

## Sugestão de correção
- `validate-room`: exigir token opaco de convite (não nome), rate limit, resposta uniforme.
- Guest: **não** migrar `organization_id`; membership isolada sem virar colega da org.
- Ou: convite guest one-time gerado pelo servidor, amarrado à sala.

## Critérios de aceite
- [ ] Sem auth, validate-room não devolve id/nome (ou exige segredo do link).
- [ ] ILIKE por nome não existe mais (ou só para membros autenticados da org).
- [ ] Guest autenticado **não** altera `users.organization_id`.
- [ ] Usuário logado de outra org continua 403 no join (já verdadeiro).

--- FIM ISSUE 3 ---
```

Issue 4 — [Segurança] Bucket de evidências público e upload sem vínculo de tenant · labels: security, severity:high

```
--- ISSUE 4 ---

# [Segurança] Bucket de evidências público e upload sem vínculo de tenant

**Labels sugeridas:** `security, severity:high`

**Achados:** F-05, F-08

## Problema
Bucket `evidence` é `public = true` e `evidence_select` é `TO public USING (bucket_id = 'evidence')`. Qualquer um lista/baixa todos os arquivos. `evidence_insert` só exige `can_write_bugs()` – sem prefixo de org/sala.

## Por que é explorável
Storage list API usa SELECT. Sem JWT. Upload: autenticado não-viewer grava em path arbitrário; URL pública (`getPublicUrl` em `src/lib/evidence.ts:52-53`).

## Evidência
`supabase/migration_access_and_security.sql:491-518`
```sql
public = true
CREATE POLICY "evidence_select" ... FOR SELECT TO public USING (bucket_id = 'evidence');
CREATE POLICY "evidence_insert" ... WITH CHECK (bucket_id = 'evidence' AND
public.can_write_bugs());
```

## Impacto
Vazamento cross-tenant de screenshots/evidências. Poluição de pastas de outras salas.

## Sugestão de correção
- Bucket **privado**. SELECT autenticado + `is_room_member(split_part(name,'/',1))` (path `roomId/uuid.ext`).
- INSERT WITH CHECK o prefixo é sala da org do caller e `is_room_member`.
- Frontend: `createSignedUrl` em vez de `getPublicUrl`.

## Critérios de aceite
- [ ] Anon não lista nem baixa objetos de `evidence`.
- [ ] Membro da sala A não lê path da sala B.
- [ ] Upload com path de outra org falha.
- [ ] App ainda exibe evidência para quem é membro (URL assinada).

--- FIM ISSUE 4 ---
```

Issue 5 — [Segurança] IDOR em UPDATE de comentários (war_room_id livre) · labels: security, severity:high

```
--- ISSUE 5 ---

# [Segurança] IDOR em UPDATE de comentários (war_room_id livre)

**Labels sugeridas:** `security, severity:high`

**Achados:** F-06, F-12

## Problema
`comments_update` só exige `user_id = auth.uid()`. Sem WITH CHECK de `is_room_member`, o autor altera `war_room_id`/`bug_id` para outra sala; os membros dela veem o comentário.

Menor: `updateBugField` usa `warRoomId` do cliente no activity log (mesmo tenant, se já for membro).

## Evidência
`supabase/migration_access_and_security.sql:413-415`
```sql
CREATE POLICY "comments_update" ON public.bug_comments
FOR UPDATE TO authenticated
USING (user_id = auth.uid());
```
`src/lib/services.ts:247-290` — log com `warRoomId` do caller.

## Impacto
Injeção de conteúdo entre salas/tenants. Integridade de auditoria no mesmo tenant (F-12).

## Sugestão de correção
```sql
USING (user_id = auth.uid() AND is_room_member(war_room_id))
WITH CHECK (user_id = auth.uid() AND is_room_member(war_room_id)
AND war_room_id IS NOT DISTINCT FROM OLD.war_room_id);
```
No client, derivar `warRoomId` do bug no servidor (não há API própria — vale trigger que copie `bugs.war_room_id`).

## Critérios de aceite
- [ ] PATCH de `war_room_id` no próprio comentário falha.
- [ ] Autor ainda edita o **texto** do comentário na sala original.
- [ ] Log de atividade não pode ser gravado numa sala da qual o user não é membro (já RLS) e o `war_room_id` bate com o do bug.

--- FIM ISSUE 5 ---
```

Issue 6 — [Segurança] Convite usa Origin do request como redirectTo · labels: security, severity:medium

```
--- ISSUE 6 ---

# [Segurança] Convite usa Origin do request como redirectTo

**Labels sugeridas:** `security, severity:medium`

**Achados:** F-07

## Problema
`inviteUserByEmail` recebe `redirectTo` montado com `req.headers.origin`. Um ator com permissão de convite aponta o magic link para um site de phishing.

## Evidência
`api-src/rooms/invite.ts:14-23`
`server.ts:115-123`
```ts
const origin = String(req.headers.origin || process.env.APP_URL ||
"https://force-qa.vercel.app");
redirectTo: `${origin.replace(/\$/ , "")}`/,
```

## Impacto
Roubo de sessão/recuperação de convite do convidado (terceiro). Requer papel QA/admin/SM.

## Sugestão de correção
Allowlist: `APP_URL` / `https://force-qa.vercel.app` / localhost só em dev. Ignorar `Origin`.

## Critérios de aceite
- [ ] Header `Origin: https://evil.example` não aparece no redirect do e-mail.
- [ ] Convite real ainda redireciona para o app de produção.

--- FIM ISSUE 6 ---
```

Issue 7 — [Segurança] Endpoints de IA sem papel, membership ou rate limit · labels: security, severity:medium


```
--- ISSUE 7 ---

# [Segurança] Endpoints de IA sem papel, membership ou rate limit

**Labels sugeridas:** `security, severity:medium`

**Achados:** F-09

## Problema
`/api/ai/suggest-bug-fields`, `detect-duplicate` e `generate-report` só chamam `requireUser`.
Viewer/guest disparam Gemini/OpenRouter. `generate-report` não valida se o caller é membro de
sala; `metrics` vem do body.

A aba Relatório IA não está escondida por papel (`WarRoomDetail.tsx:504-514`). A criação de card
sim (canWriteBugs).

## Evidência
`api-src/ai/generate-report.ts:200-203`
`api-src/ai/suggest-bug-fields.ts:12`
`api-src/ai/detect-duplicate.ts:12`
`server.ts:131-162`

## Impacto
Custo de API / DoS financeiro. Não é leitura de outro tenant no Postgres.

## Sugestão de correção
- Exigir `can_write_bugs` ou admin/QA; recusar `is_guest`.
- Rate limit por `user.id` / org.
- Relatório: receber `roomId`, carregar métricas no servidor com service_role **só se**
  `is_room_member`.

## Critérios de aceite
- [ ] JWT de viewer/guest recebe 403 nas três rotas.
- [ ] QA membro da sala continua 200.
- [ ] generate-report com roomId de outra org retorna 403.

--- FIM ISSUE 7 ---
```

Issue 8 — [Segurança] URL de evidência em href sem allowlist de esquema · labels: security, severity:medium

```
--- ISSUE 8 ---

# [Segurança] URL de evidência em href sem allowlist de esquema

**Labels sugeridas:** `security, severity:medium`

**Achados:** F-10

## Problema
`BugDetailModal` usa `href={activeBug.evidenceUrl}` para links não-imagem. `javascript:` não é filtrado. Sem sanitizer no projeto. React escapa texto, não URLs.

## Evidência
`src/components/BugDetailModal.tsx:390-392`
`src/lib/evidence.ts:13-28` (`evidenceLabel` cai em `"link"`).
`src/lib/services.ts:267` grava `evidence_url` sem validar esquema.

## Impacto
XSS armazenado no tenant para quem clica o link (precisa de papel com escrita em bugs).

## Sugestão de correção
Allowlist `https:` (e paths do Storage). Recusar `javascript:`, `data:`, `file:`. Constraint CHECK no Postgres. `rel="noopener noreferrer"` já está.

## Critérios de aceite
- [ ] Card com `evidence_url=javascript:alert(1)` não vira ``.
- \[ \] Links `https://` legítimos continuam abrindo.
- \[ \] INSERT/UPDATE com esquema inválido falha no banco ou no client.

--- FIM ISSUE 8 ---
```

Issue 9 — [Segurança] GRANT anon em organizations e schema.sql legado com USING(true) · labels: security, severity:low

--- ISSUE 9 ---

[Segurança] GRANT anon em organizations e schema.sql legado com USING(true)

****Labels sugeridas:**** `security, severity:low`

****Achados:**** F-11

Problema

`GRANT ... ON organizations TO anon` é desnecessário: policies são `TO authenticated`.
`schema.sql` (passo 1 do run order) cria policies `USING (true)` em bugs/war_rooms – inseguro se alguém aplicar ****só**** esse arquivo.

Evidência

`supabase/migration\_create\_organization\_admin.sql:4`
`supabase/schema.sql` (policies iniciais USING true)
`supabase/00\_run\_order.sql` lista schema.sql depois overlay de migrations 8–13.

Impacto

Baixo com o run order completo. Risco operacional (ambiente montado pela metade).

Sugestão de correção

- `REVOKE ALL ON public.organizations FROM anon;`
- Comentário explícito no topo de `schema.sql`: não usar em produção sem as migrations de isolamento.
- Ideal: policies iniciais já restritas ou schema.sql sem policies (só DDL).

Critérios de aceite

- [] `dp organizations` não lista anon com DML.
- [] Ambiente que rode apenas schema.sql não fica com SELECT público (ou o README proíbe esse atalho de forma inequívoca + CI aplica o run order).

--- FIM ISSUE 9 ---